

→ Methods in java :-

A method in java is a **block of code** that performs a specific task. It is executed only when it is called.

Methods help make programs modular, reusable, and easier to maintain.

~~Ex:~~

~~class~~

→ Types of methods :-

There are 4 types of methods :

Type:1.

No input

void add()

No output

Type:2.

No input

int add()

Output

Type:3.

Input

void add(int a, int b)

No output

Type:4.

Input

int add(int a, int b)

Output

Syntax :-

Return-type method-name (parameter)

{

// method body

return value ; // only for non-void methods

}

Eg; Type 1 :- Methods with no parameters & no return value

JVM

```
class calculator {
```

```
    int a;
```

```
    int b;
```

```
    int sum;
```

```
    void add() { // method declaration
```

```
        a = 10;
```

```
        b = 15;
```

```
        sum = a + b;
```

```
        System.out.println("Sum = " + sum);
```

```
    }
```

```
}
```

```
class main pogo {
```

```
    JVM {
```

```
        Calculator calci = new calculator();
```

```
        calci.add(); // method call
```

```
    }
```

```
}
```


Type 2:-

Method with no parameter but with a return value.

Class calculator {

int a;

int b;

int add()

{

a = 5;

b = 10;

return a+b;

}

}

class Pogo {

Psvm

{

Calculator calci = new Calculator ();

calci.add();

System.out.println("sum = " + calci.add());

}

}

JVM

Type 3:-

Method with parameter and no return value.

Class calculator {

```
void add(int a, int b)
{
    int sum = a + b;
    sout("sum = " + sum);
}
```

formal parameter
or
parameter

Class pogo {

psvm

{

```
Calculator calci = new Calculator();
int x = 5;
int y = 10;
calci.add(x, y);
```

}

}

Actual parameter
or
Argument

Type → 4

Method with parameter and a return value

Class calculator {

```
int add(int a, int b)
{
    return a+b;
}
```

}

class pogo {

```
    p sum
{
```

```
    Calculator calci = new calculator();
```

```
calculator int x=5;
    int y=10;
```

```
    calci.add(x,y);
```

```
    Sout (" Sum = ", + calci.add(x,y));
```

```
}
```

```
}
```

→ Method Overloading in java :-

Method overloading is a feature in java where two or more method have the same name but different parameter lists in the same class.

It is also known as Compile-Time Polymorphism because the compiler decides which method to call based on the argument.

• Rules of method overloading :-

(i) Method name must be the same.

(ii) parameter must be different by :

- Number of parameters
- Types of parameters
- Order of parameters

{Note :- changing only the return type is not overloading}

(i) Number of parameter → Method overloading :-

Class calculator {

```
void add(int a, int b) { // method declaration
    int sum = a + b;
    sout("sum=" + sum);
}
```

```
void add(int a, int b, int c) { // method declaration
    int sum = a + b + c;
    sout("sum=" + sum);
}
```


$\text{add}(10.5, 20.5)$ treat as double
 $\text{add}(10.5f, 20.5f)$ treat as float.

MON TUE WED THU FRI SAT SUN
 [] [] [] [] [] [] []

Page No.: _____

Date: ____/____/____

```
class pogo {
```

```
    psvm {
```

```
        Calculator calci = new Calculator();
```

```
        int x = 10;
```

```
        int y = 20;
```

```
        int z = 30;
```

```
        calci.add(x, y); // method call
```

```
        calci.add(x, y, z); // method call
```

```
    }
```

```
}
```

// output → sum = 30
sum = 60

(ii) Overloading by data type:-

```
Class calculator {
```

```
    void add(int a, int b) {
```

```
        int sum = a + b;
```

```
        sout("sum = " + sum);
```

```
    }
```

```
    void add(float a, float b) {
```

```
        float sum = a + b;
```

```
        sout("sum = ", sum);
```

```
    }
```

```
}
```

```
class pogo {
```

```
    psum {
```

```
        calculator calci = new calculator();
```

```
        int x1 = 5;
```

```
        int y1 = 10;
```

```
        float x2 = 15.5;
```

```
        float y2 = 20.5;
```

```
        calci.add(x1, y1);
```

```
        calci.add(x2, y2);
```

```
    }
```

```
}
```

// output is

Sum = 15

// 5 + 10

Sum = 36

// 15.5 + 20.5

What if

```
calci.add(x1, y2)
```

```
calci.add(x2, y1)
```

Sum = 25.5

// 5 + 20.5

Sum = 25.5

// ~~15.5~~ 15.5 + 10

{ Isme dono case void add(float a, float b) के जया !
 क्योंकि compiler function call के अंदर के data type
 देख कर function declare करती है }

(iii) Overloading by order of parameter :-

Class calculator {

void ~~sum~~ add(double a, int b) {

double sum = a+b;

Sout("sum = " + sum);

}

void add(int a, double b) {

double sum = a+b;

Sout("sum = " + sum);

}

}

Class pugo {

psum {

calculator calci = new calculator();

calci.add(10.5, 25);

calci.add(20, 5.5);

}

}

→ How does compiler choose the method?

(i) `calci.add(10, 20);`

Compiler searches for

`add(int, int)`

(ii) `calci.add(10.5, 20.5);`

compiler searches for

`add(double, double)`

(iii) `calci.add(20, 5.5);`

Compiler searches for

`add(int, double)`

∴ Changing only the return type is not overloading

class ~~calculator~~ calculator {

int add (int a, int b) {

return a+b; }

double add(int a, int b) {

return a+b; }

The parameter list is the same, so java cannot distinguish the method.

Akash